

BFS vs DFS in State Space Exploration

These are two fundamental strategies for traversing a graph or tree of states (nodes), each with distinct tradeoffs.

How They Work

BFS explores all neighbors of a node before moving deeper. It uses a **queue** (FIFO) and expands level by level.

DFS explores as far down a branch as possible before backtracking. It uses a **stack** (LIFO) — either explicit or via recursion.

Key Properties

Property	BFS	DFS
Data structure	Queue	Stack / Recursion
Completeness	✅ Complete (finds solution if one exists)	✅ Complete on finite graphs
Optimality	✅ Optimal (shortest path, uniform cost)	❌ Not guaranteed
Time complexity	$O(b^d)$	$O(b^m)$
Space complexity	$O(b^d)$ — stores all frontier nodes	$O(b \cdot m)$ — stores one path at a time
Best for	Shallow solutions, shortest paths	Deep solutions, memory-constrained

b = branching factor, d = solution depth, m = max depth

The Core Tradeoff

BFS's killer weakness is memory. It must hold the entire frontier in memory. In a state space with branching factor 10 and solution at depth 10, that's 10 billion nodes. It's impractical for large search spaces.

DFS's killer weakness is optimality. It can dive down a very long (or infinite) path and miss a shallow solution entirely. On infinite graphs without a depth limit, it may never terminate.